

Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
alekhya@Alekhyareddy: ~  
alekhya@Alekhyareddy:~$ nano main_loop.c  
alekhya@Alekhyareddy:~$ gcc main_loop.c -o main_loop  
alekhya@Alekhyareddy:~$ ./main_loop  
myshell> hello  
You entered: hello  
myshell> linux  
You entered: linux  
myshell> exit  
Exiting shell...  
alekhya@Alekhyareddy:~$ |
```

```
GNU nano 7.2  
#include <stdio.h>  
#include <string.h>  
  
int main()  
{  
    char command[100];  
    while (1)  
    {  
        printf("myshell> ");  
        fgets(command, sizeof(command), stdin);  
        command[strcspn(command, "\n")] = '\0';  
        if (strcmp(command, "exit") == 0)  
        {  
            printf("Exiting shell...\n");  
            break;  
        }  
        printf("You entered: %s\n", command);  
    }  
    return 0;  
}
```

Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: open(" filename.txt", modes, permissions)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

#include <unistd.h>

```
int main()
{
    char buffer[100];
    int n;
    printf("Enter some text: ");
    n = read(0, buffer, sizeof(buffer) - 1);
    buffer[n] = '\0'; // Null-terminate the string
    printf("You entered: %s", buffer);
    return 0;
}
```

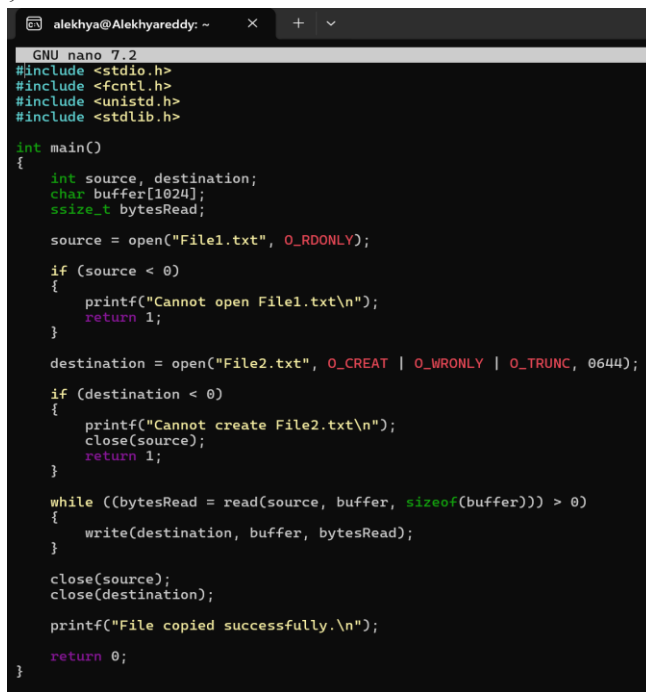
write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

```
#include <unistd.h>

int main()
{
    write(1, "Hello World\n", 12);
    return 0;
}
```



```
GNU nano 7.2
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead;

    source = open("File1.txt", O_RDONLY);
    if (source < 0)
    {
        printf("Cannot open File1.txt\n");
        return 1;
    }

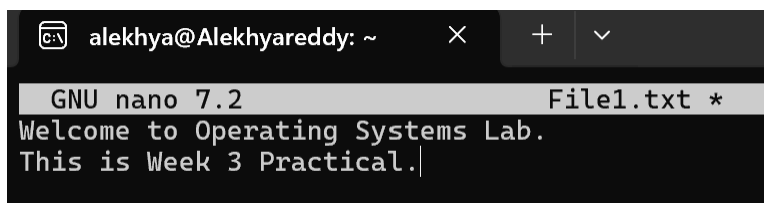
    destination = open("File2.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
    if (destination < 0)
    {
        printf("Cannot create File2.txt\n");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        write(destination, buffer, bytesRead);
    }

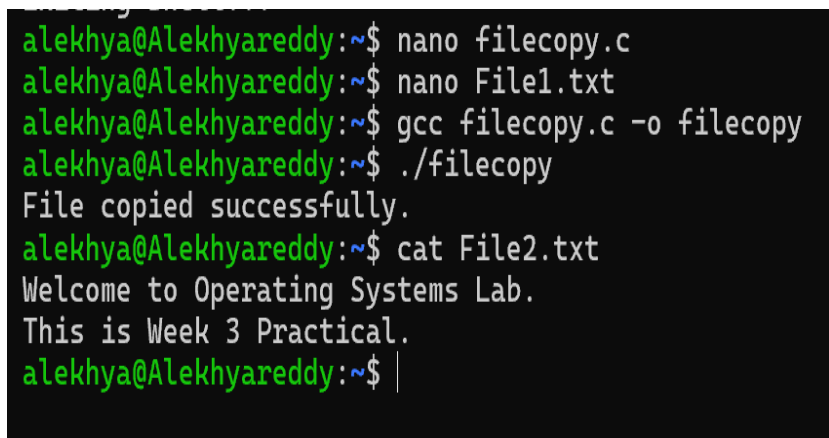
    close(source);
    close(destination);

    printf("File copied successfully.\n");

    return 0;
}
```

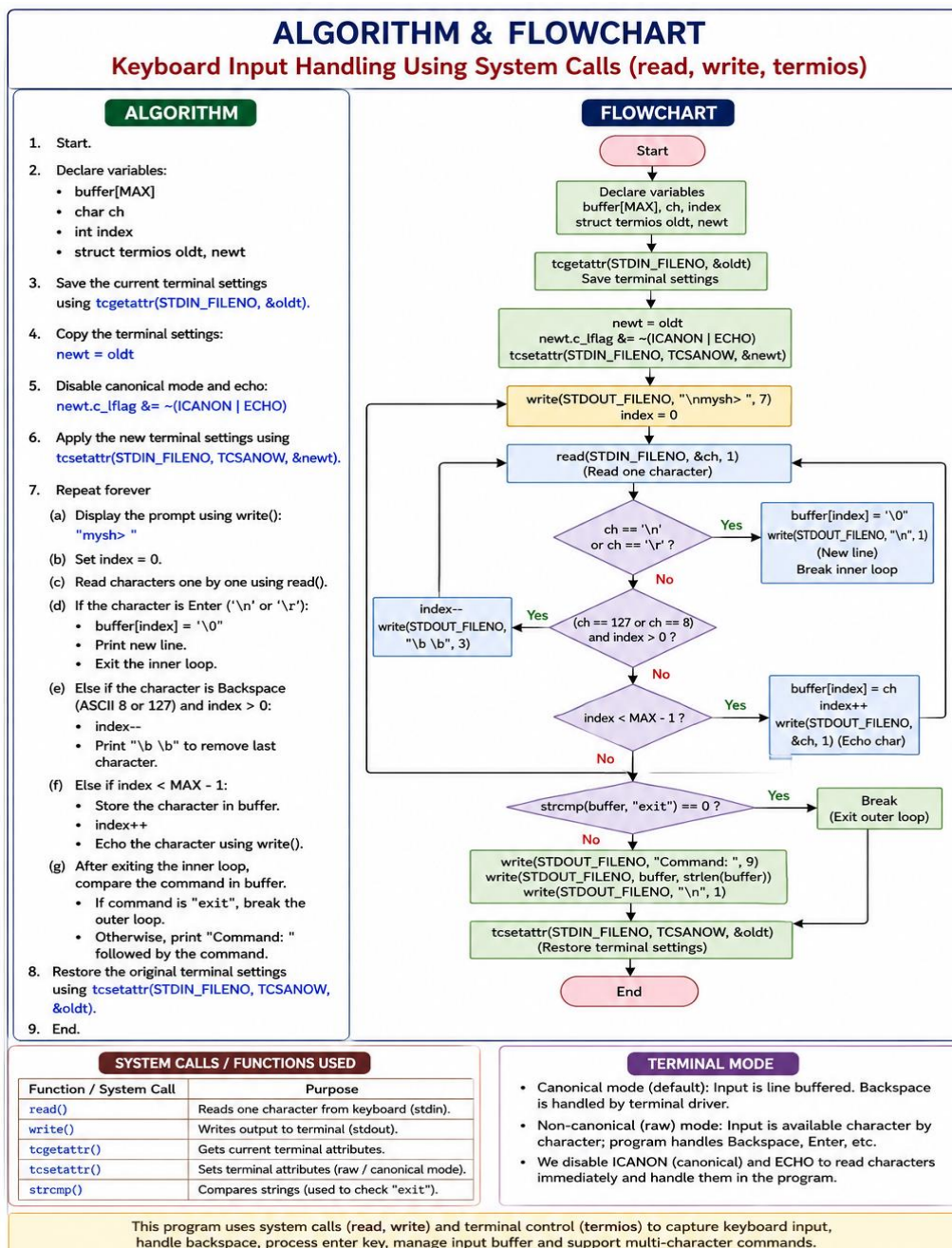


```
GNU nano 7.2 File1.txt *
Welcome to Operating Systems Lab.
This is Week 3 Practical.
```



```
alekhya@AlekhyaReddy:~$ nano filecopy.c
alekhya@AlekhyaReddy:~$ nano File1.txt
alekhya@AlekhyaReddy:~$ gcc filecopy.c -o filecopy
alekhya@AlekhyaReddy:~$ ./filecopy
File copied successfully.
alekhya@AlekhyaReddy:~$ cat File2.txt
Welcome to Operating Systems Lab.
This is Week 3 Practical.
alekhya@AlekhyaReddy:~$
```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.



GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define BUFFER_SIZE 100

void enableRawMode(struct termios *original)
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);

    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void disableRawMode(struct termios *original)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

int main()
{
    struct termios original;
    char buffer[BUFFER_SIZE];
    int length = 0;

    enableRawMode(&original);

    printf("myshell> ");
    fflush(stdout);

    while (1)
    {
```

```
alekhya@Alekhyareddy: ~/OS × + v
GNU nano 7.2
char c;

if (read(STDIN_FILENO, &c, 1) != 1)
    continue;

/* Enter key */
if (c == '\n' || c == '\r')
{
    buffer[length] = '\0';

    printf("\nYou entered: %s\n", buffer);

    if (strcmp(buffer, "exit") == 0)
    {
        break;
    }

    length = 0;
    memset(buffer, 0, BUFFER_SIZE);

    printf("myshell> ");
    fflush(stdout);
}

/* Backspace */
else if (c == 127 || c == 8)
{
    if (length > 0)
    {
        length--;

        buffer[length] = '\0';

        printf("\b \b");
        fflush(stdout);
    }
}
```

```

}

/* Normal character */
else if (length < BUFFER_SIZE - 1)
{
    buffer[length++] = c;

    putchar(c);
    fflush(stdout);
}

disableRawMode(&original);

printf("\nExiting shell...\n");

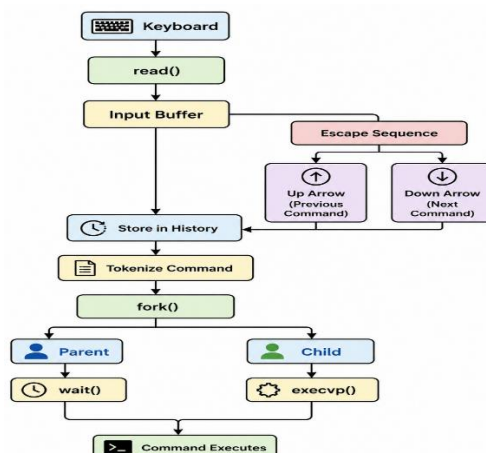
return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut
^X Exit      ^R Read File  ^\ Replace    ^U Paste
```

```
alekhya@Alekhyareddy: ~/OS × + ▾
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano task3_input.c
alekhya@Alekhyareddy:~/OSSP$ gcc task3_input.c -o task3_input
alekhya@Alekhyareddy:~/OSSP$ ./task3_input
myshell> hello world
You entered: hello world
myshell> ./task3_input
You entered: ./task3_input
myshell>
You entered:
myshell> helo
You entered: helo
myshell> ./task3_input
You entered: ./task3_input
myshell> exit
You entered: exit

Exiting shell...
alekhya@Alekhyareddy:~/OSSP$ |
```

Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality



GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define BUFFER_SIZE 100
#define HISTORY_SIZE 10

char history[HISTORY_SIZE][BUFFER_SIZE];
int history_count = 0;
int history_position = 0;

void enable_raw_mode(struct termios *original)
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);

    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void disable_raw_mode(struct termios *original)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

void clear_line(int length)
{
    for (int i = 0; i < length; i++)
        printf("\b \b");

    fflush(stdout);
}

void add_history(const char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count < HISTORY_SIZE)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 0; i < HISTORY_SIZE - 1; i++)
        {
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

^T
^J

```
    strcpy(history[history_count], command);
    history_count++;
}
else
{
    for (int i = 0; i < HISTORY_SIZE - 1; i++)
    {
        strcpy(history[i], history[i + 1]);
    }

    strcpy(history[HISTORY_SIZE - 1], command);
}

history_position = history_count;
}

int main()
{
    struct termios original;
    char buffer[BUFFER_SIZE];

    int length = 0;

    enable_raw_mode(&original);

    printf("myshell> ");
    fflush(stdout);

    while (1)
    {
        char c;

        if (read(STDIN_FILENO, &c, 1) != 1)
            continue;

        /* Enter key */
        if (c == '\n' || c == '\r')
        {
            buffer[length] = '\0';

            printf("\n");

            if (strcmp(buffer, "exit") == 0)
            {
                break;
            }

            if (length > 0)
            {
                printf("Command: %s\n", buffer);
                add_history(buffer);
            }
        }
    }
}
```



alekhya@Alekhyareddy: ~/OS



GNU nano 7.2

```
        length = 0;
        buffer[0] = '\0';

        printf("myshell> ");
        fflush(stdout);
    }

    /* Backspace */
    else if (c == 127 || c == 8)
    {
        if (length > 0)
        {
            length--;
            buffer[length] = '\0';

            printf("\b \b");
            fflush(stdout);
        }
    }

    /* Escape sequence */
    else if (c == 27)
    {
        char seq[2];

        if (read(STDIN_FILENO, &seq[0], 1) != 1)
            continue;

        if (seq[0] != '[')
            continue;

        if (read(STDIN_FILENO, &seq[1], 1) != 1)
            continue;

        /* Up Arrow */
        if (seq[1] == 'A')
        {
            if (history_count > 0 && history_position > 0)
            {
                clear_line(length);

                history_position--;

                strcpy(buffer, history[history_position]);
                length = strlen(buffer);

                printf("%s", buffer);
                fflush(stdout);
            }
        }

        /* Down Arrow */
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^W Where Is
^_ Replace
```

```
^K Cut
^U Paste
```

```
^T Execute
^J Justify
```

```
^C L
^_ G
```

```
        length = strlen(buffer);

        printf("%s", buffer);
        fflush(stdout);
    }
}

/* Down Arrow */
else if (seq[1] == 'B')
{
    if (history_position < history_count - 1)
    {
        clear_line(length);

        history_position++;

        strcpy(buffer, history[history_position]);
        length = strlen(buffer);

        printf("%s", buffer);
        fflush(stdout);
    }
    else
    {
        clear_line(length);

        history_position = history_count;
        buffer[0] = '\0';
        length = 0;

        fflush(stdout);
    }
}

/* Normal character */
else if (length < BUFFER_SIZE - 1)
{
    buffer[length++] = c;
    buffer[length] = '\0';

    putchar(c);
    fflush(stdout);
}

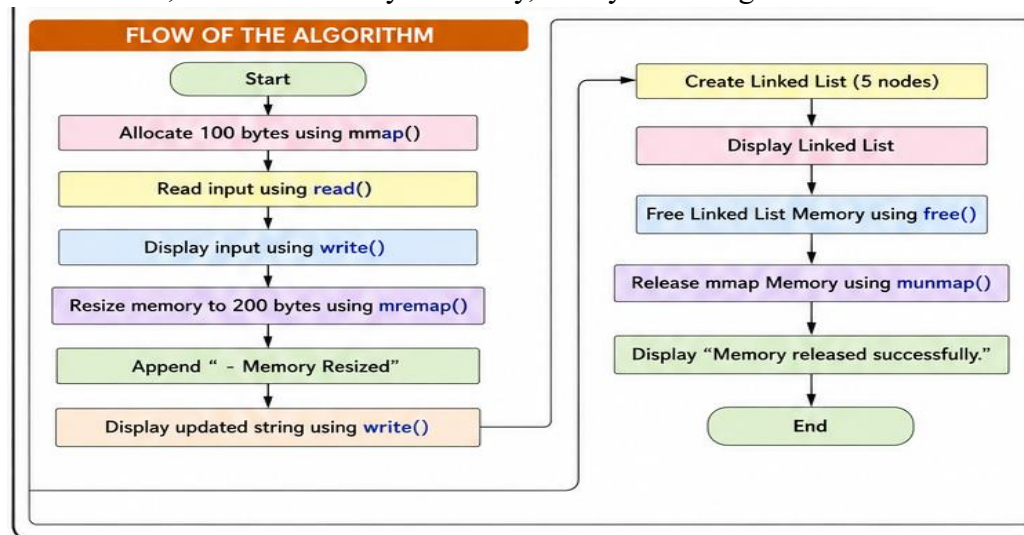
disable_raw_mode(&original);

printf("\nExiting shell...\n");

return 0;
}
```

```
alekhya@Alekhyareddy: ~/OS × + v
alekhya@Alekhyareddy:~$ cd ~/OSSP
alekhya@Alekhyareddy:~/OSSP$ nano task4_history.c
alekhya@Alekhyareddy:~/OSSP$ gcc task4_history.c -o task4_history
alekhya@Alekhyareddy:~/OSSP$ ./task4_history
myshell> hello
Command: hello
myshell> linux
Command: linux
myshell> test
Command: test
myshell> hello
Command: hello
myshell> test
Command: test
myshell> linux
Command: linux
myshell> exit
Exiting shell...
```

Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.



Validate the program with valgrind tool

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

struct Node
{
    char *command;
    struct Node *next;
};

/* Add command to linked list */
void add_command(struct Node **head, const char *command)
{
    struct Node *newNode;

    newNode = malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->command = malloc(strlen(command) + 1);

    if (newNode->command == NULL)
    {
        free(newNode);
        printf("Memory allocation failed!\n");
        exit(1);
    }

    strcpy(newNode->command, command);

    newNode->next = *head;
    *head = newNode;
}

/* Display linked list */
void display_commands(struct Node *head)
{
    int count = 1;

    printf("\nCommand History:\n");
```

```
^G Help
^X Exit
```

```
^O Write Out
^R Read File
```

```
^W Where Is
^_ Replace
```

```
^K Cut
^U Paste
```

GNU nano 7.2

```
while (head != NULL)
{
    printf("%d. %s\n", count, head->command);

    head = head->next;
    count++;
}

/* Release allocated memory */
void free_commands(struct Node *head)
{
    struct Node *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;

        free(temp->command);
        free(temp);
    }
}

int main()
{
    char *buffer;
    int size = INITIAL_SIZE;
    int length = 0;
    int ch;

    struct Node *history = NULL;

    buffer = malloc(size);

    if (buffer == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("Dynamic Buffer and Linked List Demo\n");
    printf("Enter a command: ");

    while ((ch = getchar()) != '\n' && ch != EOF)
    {
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

GNU nano 7.2

```
    if (length >= size - 1)
    {
        size *= 2;

        char *temp = realloc(buffer, size);

        if (temp == NULL)
        {
            free(buffer);
            free_commands(history);

            printf("\nMemory reallocation failed!\n");
            return 1;
        }

        buffer = temp;

        printf("\nBuffer resized to %d bytes\n", size);
    }

    buffer[length++] = (char)ch;
}

buffer[length] = '\0';

printf("\nEntered command: %s\n", buffer);
printf("Buffer size: %d bytes\n", size);

/* Store command in linked list */
add_command(&history, buffer);

/* Add some sample commands */
add_command(&history, "ls");
add_command(&history, "pwd");
add_command(&history, "gcc program.c");

display_commands(history);

/* Release memory */
free(buffer);
free_commands(history);

printf("\nMemory released successfully.\n");

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^W Where Is
^_ Replace

^K Cut
^U Paste

alekhya@Alekhyareddy: ~/OS × + ▾

```
alekhya@Alekhyareddy:~$ cd ~/OSSP
```

```
alekhya@Alekhyareddy:~/OSSP$ nano task5_memory.c
```

```
alekhya@Alekhyareddy:~/OSSP$ gcc -Wall -Wextra task5_memory.c -o task5_memory
```

```
alekhya@Alekhyareddy:~/OSSP$ ./task5_memory
```

Dynamic Buffer and Linked List Demo

Enter a command: this_is_a_very_long_command_to_test_dynamic_buffer_resizing

Buffer resized to 20 bytes

Buffer resized to 40 bytes

Buffer resized to 80 bytes

Entered command: this_is_a_very_long_command_to_test_dynamic_buffer_resizing

Buffer size: 80 bytes

Command History:

1. gcc program.c

2. pwd

3. ls

4. this_is_a_very_long_command_to_test_dynamic_buffer_resizing

Memory released successfully.

```
alekhya@Alekhyareddy:~/OSSP$ valgrind --version
```

valgrind-3.22.0

```
alekhya@Alekhyareddy:~/OSSP$ valgrind --leak-check=full ./task5_memory
```

==5075== Memcheck, a memory error detector

==5075== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.

==5075== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info

==5075== Command: ./task5_memory

==5075==

Dynamic Buffer and Linked List Demo

Enter a command: hello this is a very long command for testing dynamic buffer resizing

Buffer resized to 20 bytes

Buffer resized to 40 bytes

Buffer resized to 80 bytes

Entered command: hello this is a very long command for testing dynamic buffer resizing

Buffer size: 80 bytes

alekhya@AlekhyaReddy: ~/OS

==5075==

Dynamic Buffer and Linked List Demo

Enter a command: hello this is a very long command for testing dynamic buffer resizing

Buffer resized to 20 bytes

Buffer resized to 40 bytes

Buffer resized to 80 bytes

Entered command: hello this is a very long command for testing dynamic buffer resizing

Buffer size: 80 bytes

Command History:

1. gcc program.c

2. pwd

3. ls

4. hello this is a very long command for testing dynamic buffer resizing

Memory released successfully.

==5075==

==5075== HEAP SUMMARY:

==5075== in use at exit: 0 bytes in 0 blocks

==5075== total heap usage: 14 allocs, 14 frees, 2,353 bytes allocated

==5075==

==5075== All heap blocks were freed -- no leaks are possible

==5075==

==5075== For lists of detected and suppressed errors, rerun with: -s

==5075== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

alekhya@AlekhyaReddy:~/OSSP\$